



पोखरा विश्वविद्यालय
POKHARA UNIVERSITY



Theory Of Computation

Code: CMP 224

6. Undecidability

Er. Nirmal Thapa

Email: nirmalthapa@lumbini.edu.np

Lumbini Engineering College

Pokhara University

Church-Turing Thesis

It is a mathematically un-provable hypothesis about the computability.

This hypothesis simply states that - "Any algorithmic procedure that can be carried out at all (by human, a team of human or a computer) can be carried out by a TM."

1. No one has yet been able to suggest a problem solvable by what we intuitively consider an algorithm, for which a TM program cannot be written
2. Alternative models have been proposed for mechanical computation, but none of them is more powerful than the TM model.

After adopting Church-Turing Thesis, we are giving a precise meaning of the term:

An algorithm is a procedure that can be executed on a Turing Machine.

Decidability and Undecidability

Recursive Language:

- A language 'L' is said to be recursive if there exists a Turing machine which will accept all the strings in 'L' and reject all the strings not in 'L'.
- The Turing machine will halt every time and give an answer (accepted or rejected) for each and every string input.

Recursively Enumerable Language:

- A language 'L' is said to be a recursively enumerable language if there exists a Turing machine which will accept (and therefore halt) for all the input strings which are in 'L'.
- But may or may not halt for all input strings which are not in 'L'.

Decidability and Undecidability

Decidable Language:

A language 'L' is decidable if it is a recursive language. All decidable languages are recursive languages and vice-versa.

Partially Decidable Language:

A language 'L' is partially decidable if 'L' is a recursively enumerable language.

Undecidable Language:

- A language is undecidable if it is not decidable.
- An undecidable language may sometimes be partially decidable but not decidable.
- If a language is not even partially decidable, then there exists no Turing machine for that language

Decidability and Undecidability

Recursive Language	TM will always Halt
Recursively Enumerable Language	TM will halt sometimes & may not halt sometimes
Decidable Language	Recursive Language
Partially Decidable Language	Recursively Enumerable Language
UNDECIDABLE	No TM for that language

A limitation of Turing Machines:

Turing Machines are "hardwired"

they execute only one program

Real Computers are re-programmable

Solution: **Universal Turing Machine**

Attributes:

- Reprogrammable machine
- Simulates any other Turing Machine

Universal Turing Machine
simulates any Turing Machine M

Input of Universal Turing Machine:

Description of transitions of M

Input string of M

Universal Turing Machine

- A Turing machine is created to execute a specific algorithm.
- Computing different function or doing some other calculation, a different TM will be required.
- Originally electronic computers were limited in a similar way, and changing the computation to be performed, requires rewriting the machine.
- A - TM can be used as a "stored program computer" taking its program as well as data from one or more tapes on which it is placed.
- Turing describes a "Universal Computing Machine" that can be defined as:
 - A Universal Turing machine (T_u) is a TM whose input consist of program and a data set for the program to process.
 - The program takes the form of a string specifying some other (special - purpose) TM T_1 and the data set is second string w interpreted as input to T_1 .
 - T_u then simulates the processing of w by T_1 .

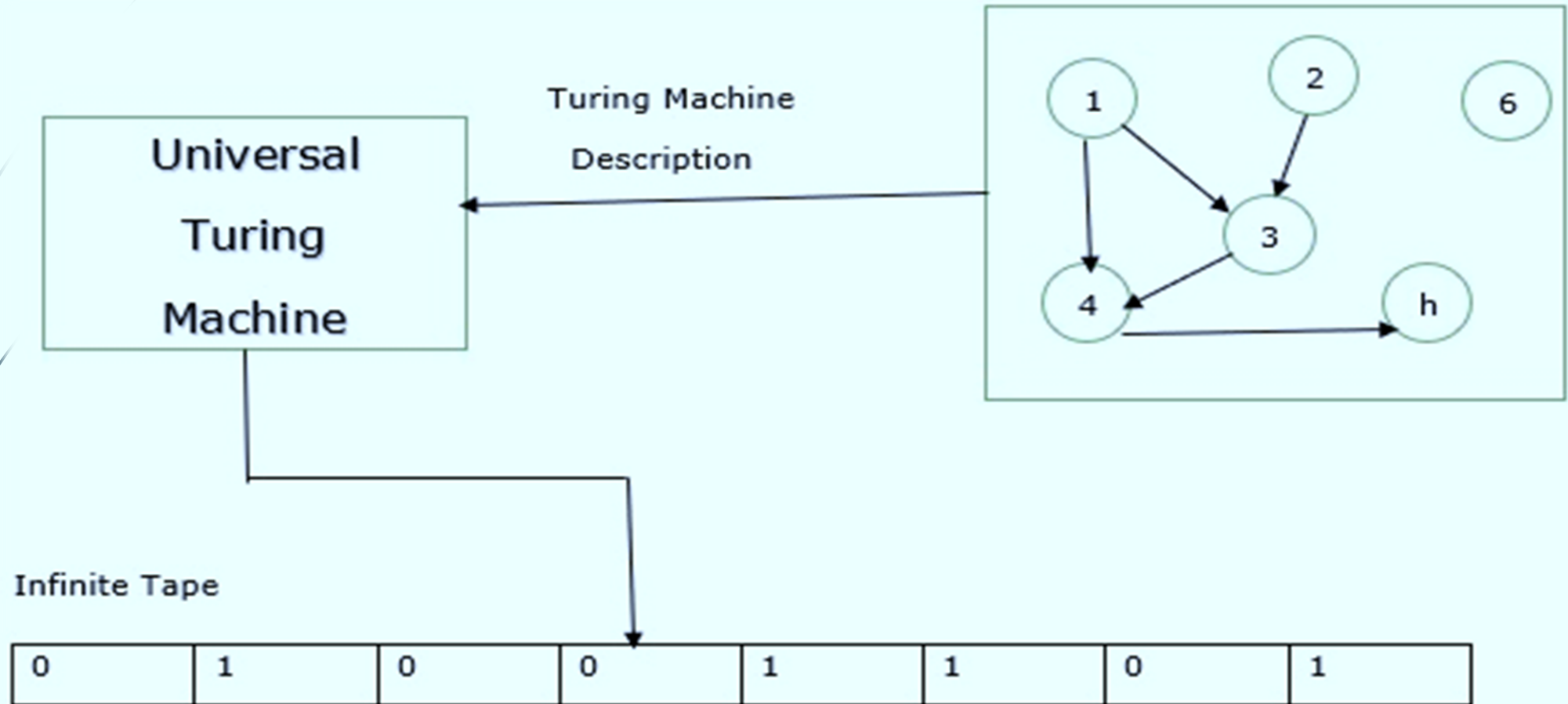
Universal Turing Machine

This machine would have three bits of information for the machine it is simulating

- A basic description of the machine.
- The contents of machine tape.
- The internal state of the machine.
- The Universal machine would simulate the machine by looking at the input on the tape and the state of the machine.
- It would control the machine by changing its state based on the input.
- This leads to the idea of a "computer running another computer".

Universal Turing Machine

The schematic diagram of the Universal Turing Machine is as follows



Three tapes

Universal
Turing
Machine

Tape 1



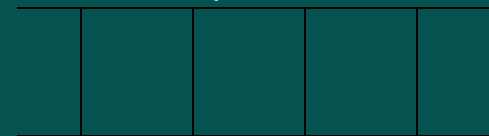
Description of M

Tape 2



Tape Contents of M

Tape 3



State of M

Tape 1

--	--	--	--	--

Description of M

We describe Turing machine M
as a string of symbols:

We encode M as a string of symbols

Universal Turing Machine

We now define the **Universal Turing Machine**, T_u .

On input (M, w) , where M is a TM and a string w

1. Checks that (M, w) is a proper encoding of a TM, followed by a string from Σ^*
2. Simulates M on input w
3. If M on input w enters its accept state, T_u **accept**, and if M on input w ever enters its reject state, T_u **reject**.

Alphabet Encoding

Symbols:

a

b

c

d

...

Encoding:

1

11

111

1111

State Encoding

States:

q_1 q_2 q_3 q_4 ...

Encoding:

$\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$

1 11 111 1111

Head Move Encoding

Move:

L R

Encoding:

$\downarrow$ $\downarrow$

1 11

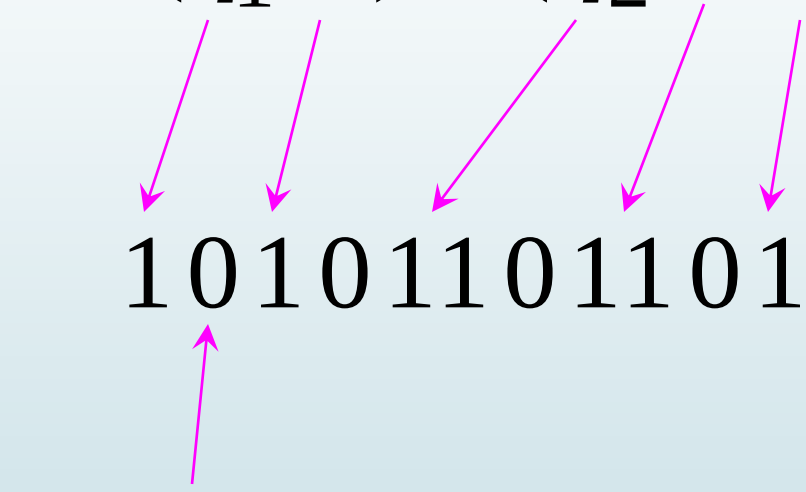
Transition Encoding

Transition: $\delta(q_1, a) = (q_2, b, L)$

Encoding:

10101101101

separator



Turing Machine Encoding

Transitions:

$$\delta(q_1, a) = (q_2, b, L) \quad \delta(q_2, b) = (q_3, c, R)$$

Encoding:

1 0 1 0 1 1 0 1 1 0 1 0 0 1 1 0 1 1 0 1 1 1 0 1 1 1 0 1 1

separator

Tape 1 contents of Universal Turing Machine:

binary encoding
of the simulated machine M

Tape 1

1 0 1 0 11 0 11 0 10011 0 1 10 111 0 111 0 1100 ...



A Turing Machine is described with a binary string of 0's and 1's

Therefore:

The set of Turing machines forms a language:

Each string of this language is the binary encoding of a Turing Machine

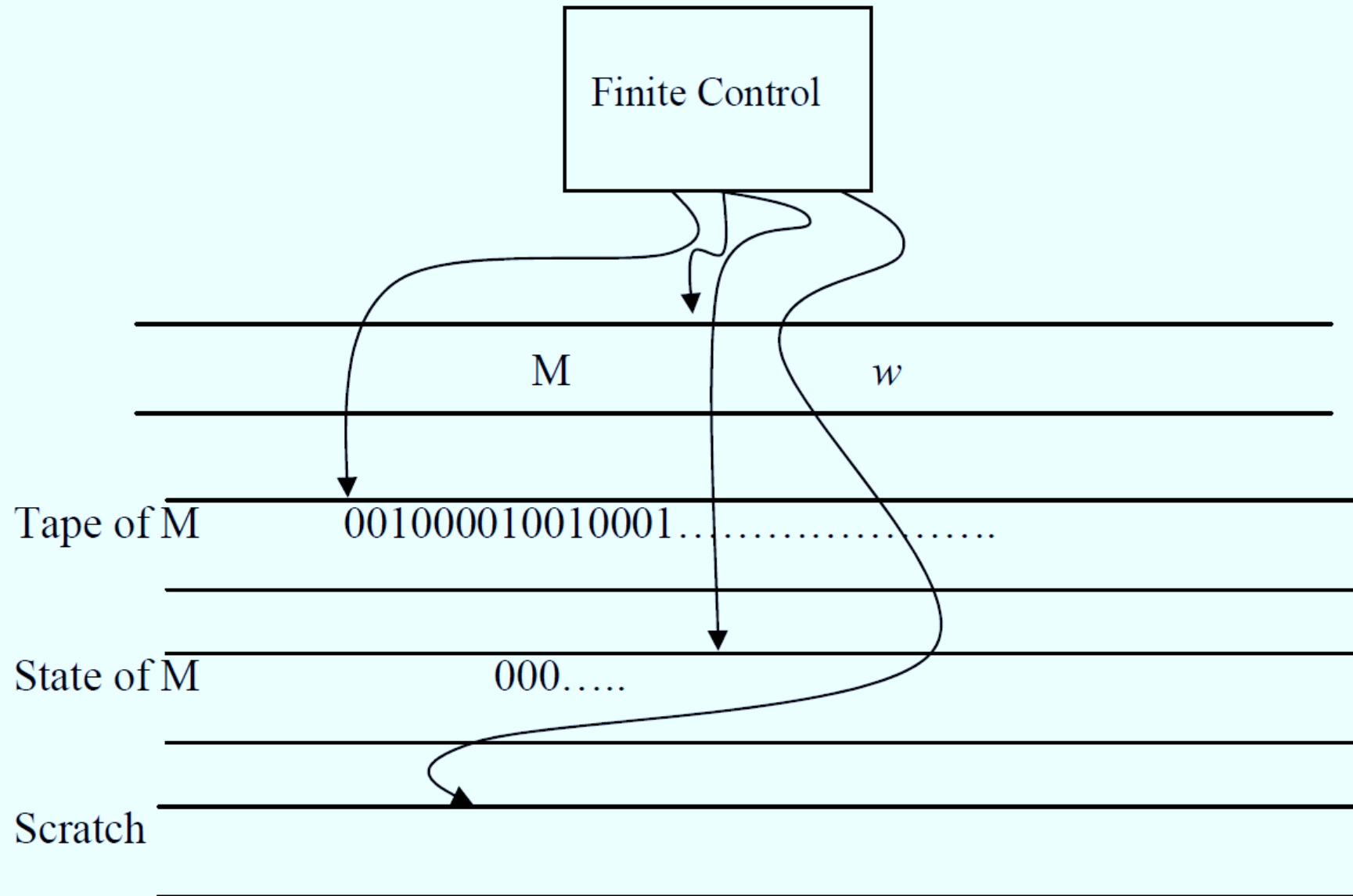
Language of Turing Machines

$L = \{$ 010100101, (Turing Machine 1)
00100100101111, (Turing Machine 2)
111010011110010101,
.....
..... }

Universal Languages

- The universal language L_u is the set of binary strings that encode a pair (M, w) where M is a TM with binary input alphabet, and w is a string in $(0+1)^*$, such that w is in $L(M)$.
- In other words, the languages that is accepted by universal TM is called universal Languages.
- A universal TM U can be described as a multi-tape Turing machine in which the transitions of M are stored initially on the first tape along with string w .
- The second tape holds the simulated tape of M in the format same as the code of M .
- Third tape of U holds the state of M , with suitable encoding as in figure below.

Universal Languages





Halting Problem

- **Given a program/algorithm will ever halt or not?**
- Halting means that the program on certain input will accept it and halt or reject it and halt and it would never go into an infinite loop.
- Basically halting means terminating.
- So can we have an algorithm that will tell that the given program will halt or not.
- In terms of Turing machine, will it terminate when run on some machine with some particular given input string?
- The answer is no we cannot design a generalized algorithm which can appropriately say that given a program will ever halt or not?

Halting Problem

- **Proof by Contradiction :**
- **Problem statement:** Can we design a machine which if given a program can find out if that program will always halt or not halt on a particular input?

Solution:

- Let us assume that we can design that kind of machine called as $HM(P, I)$
- If we can design such a program this allows us to write another program we call this program $CM(X)$

Halting Problem

HM (P,I)

{ Halt

or

May not Halt

}

CM (X)

{

if(HM(X,X)==Halt)
loop forever;

else
return;

}

Halting Problem

HM (P,I)

```
{ Halt
```

or

```
May not Halt
```

```
}
```

CM (X)

```
{
```

```
  if ( H (X,X) ==HALT)
```

```
    loop forever;
```

```
  else
```

```
    return;
```

```
}
```

if we run CM on itself:

CM(CM,CM)

HM(CM,CM) ==HALT

```
{    Loop forever;  
  // it means it will never halt;  
}
```

H(C,C) ==NOT HALT

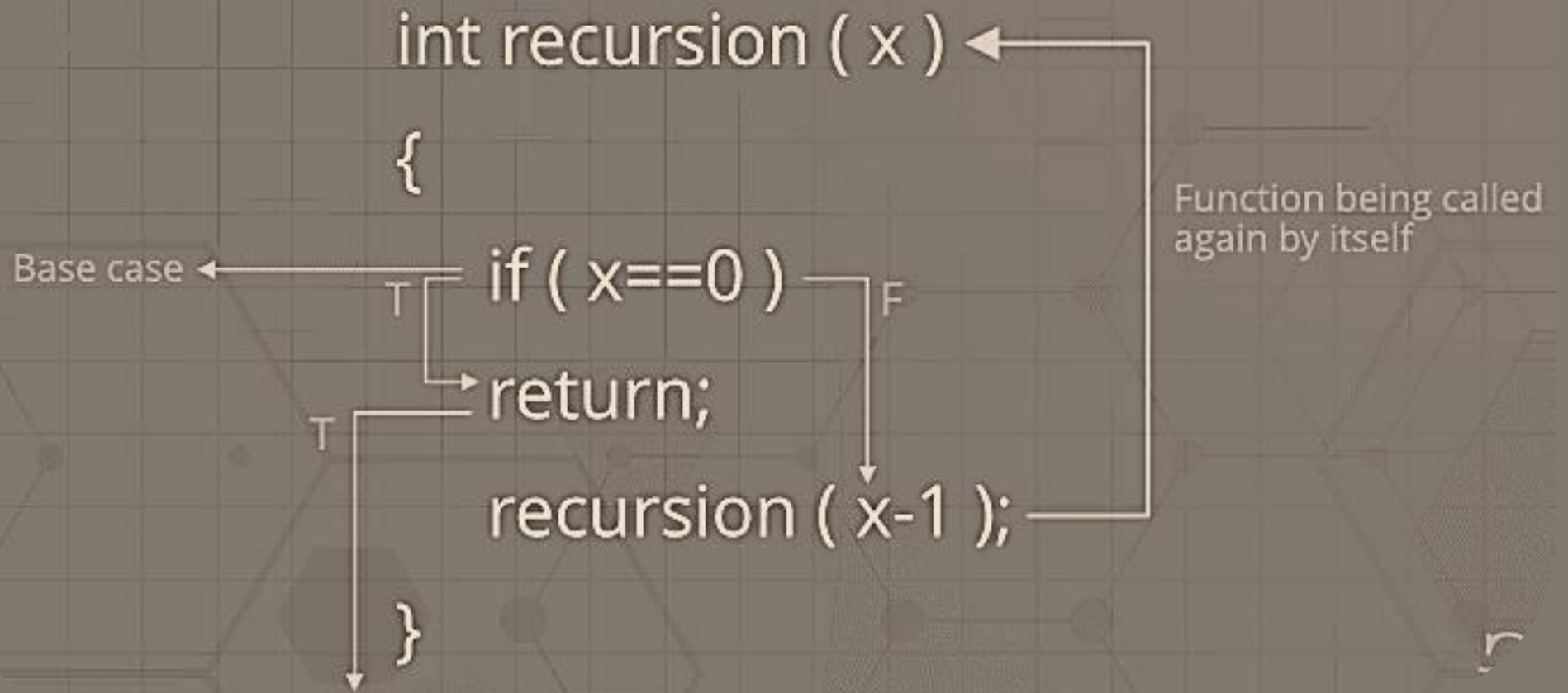
```
{    Halt;  
  // it will never halt because of the  
  // above non-halting condition;  
}
```


Recursive Function Theory

- A function that calls itself directly or indirectly is called a recursive function.
- The recursive factorial function uses more memory than its non-recursive counter part.
- Recursive function requires stack support to save the recursive function calls.
 - It is simple
 - It is easy to understand
 - It is easy to prove correct

Recursive Function Theory

Recursive Functions



Recursive Function Theory

Functions

- The concept of a function is a fundamental topic in mathematics.
- A function or a total function $f: X \rightarrow Y$ is a rule that assigns to all the elements of one set, X , a unique element of another set, Y .
- The first set, X is called the domain of the function, and the second set, Y is called its range.

Recursive Function Theory

Total Function

A total function from X to Y is a rule that assigns to every element of X , a unique element of Y .

$$f : X \rightarrow Y$$

For example, $f(x) = 2x$

Partial Function

Here, to extend the class of computable functions, we use partial functions.
A partial function, $f: X \rightarrow Y$

Recursive Function Theory

Primitive Recursive Functions

A function, f is called a primitive recursive function,

- i) If it is one of the **three basic functions**, or,
- ii) If it can be obtained by applying operations such as composition and recursion to the set of basic functions.

The basic functions and operations are explained below:

1. Basic Functions

- Zero Function

$Z(x) = 0$ is called Zero function. Example: We may write, $Z(8) = 0$

2. Successor Function

The successor function is $S(x)$, defined as $S(x) = x+1$

Thus the value of $S(x)$ is the integer next in sequence to x .

Recursive Function Theory

3. Projection Function

Projection function, k is defined as,

$$P_k(x_1, x_2, \dots, x_k, \dots, x_n) = x_k$$

For example,

$$P_3(8, 5, 6, 4) = 6$$

$$P_6(6, 34, 7, 2, 45, 23, 22) = 23$$

Recursive Function Theory

Composition of Function:

A composite function is defined when one function is substituted into another function

For example,

Let $f(x) = 3x + 2$ and $g(x) = x + 5$

Then $f(g(x)) = ?$ Or

$g(f(x)) = ?$

Recursive Function Theory

Partial Recursive Functions

A function, f is a partial recursive function if either,

- i. it is one of the **basic functions**, or
- ii. it is produced by performing operations such as composition, recursion and minimization to the basic functions

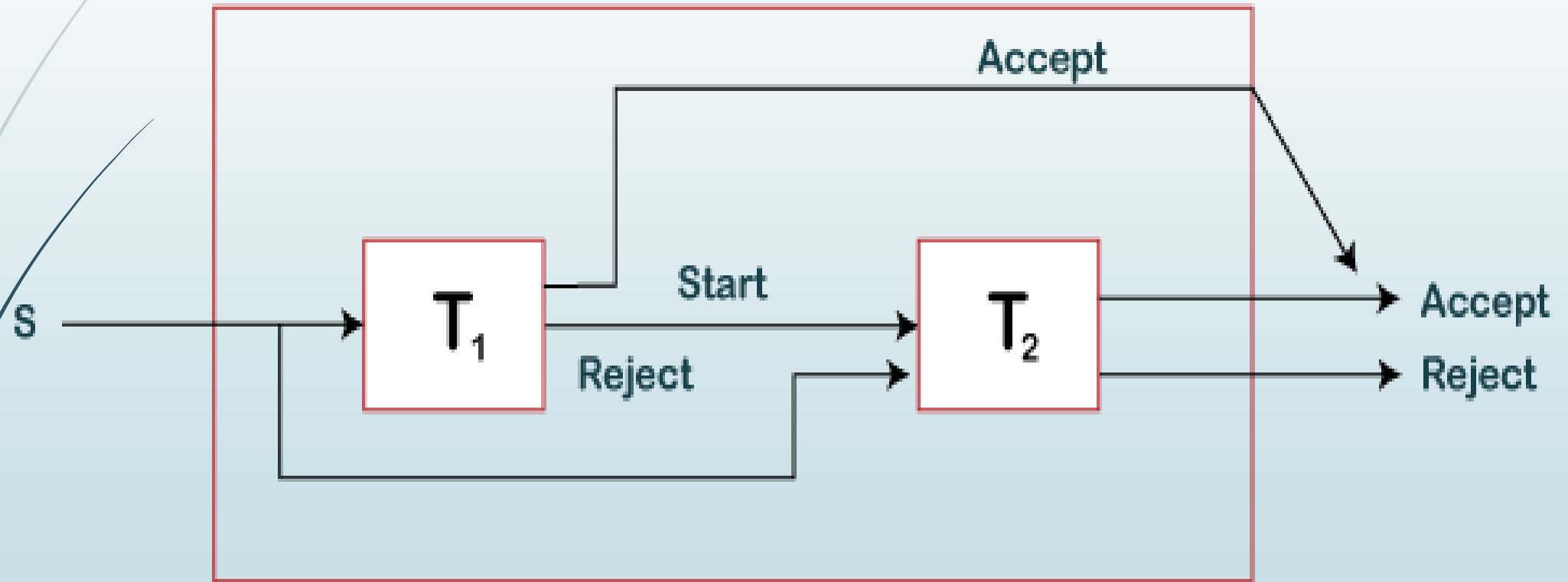
Properties of Recursive and RELs

Union:

- If L_1 and L_2 are two recursive languages, their union $L_1 \cup L_2$ will also be recursive
- Because if TM halts for L_1 and halts for L_2 , it will also halt for $L_1 \cup L_2$.
- Let's revise union of sets;
- Set 1 = {a, b, c}
- Set 2 = {b, c, d}
- Set 1 Union Set 2 = {a, b, c, d}

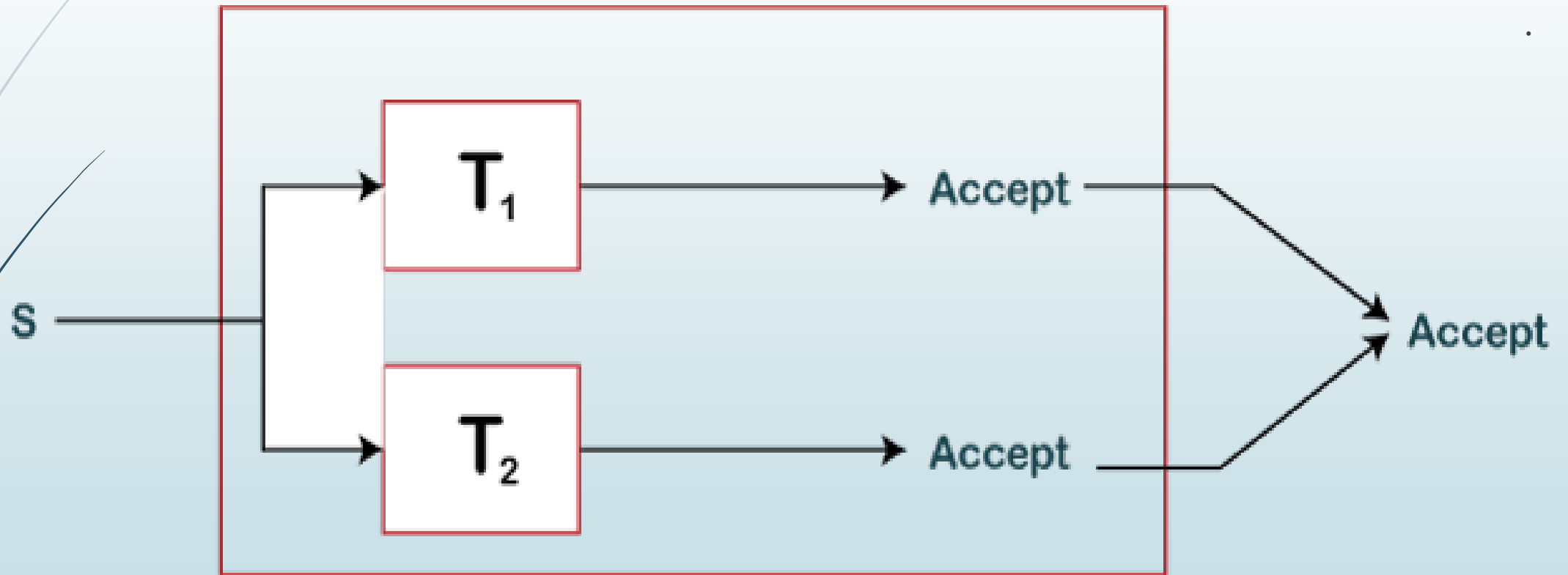
Properties of Recursive and RELs

Union: If L_1 and L_2 are two recursive languages then the union of L_1 and L_2 i.e. $L_1 \cup L_2$ is also recursive.



Properties of Recursive and RELs

Union: The union of any two recursively enumerable language is also recursively enumerable language



Properties of Recursive and RELs

Concatenation: If $L1$ and $L2$ are two recursive languages, their concatenation $L1.L2$ will also be recursive.

For Example:

$$L1 = \{a^n b^n c^n \mid n \geq 0\}$$

$$L2 = \{d^m e^m f^m \mid m \geq 0\}$$

$$L3 = L1.L2$$

$= \{a^n b^n c^n d^m e^m f^m \mid m \geq 0, n \geq 0\}$ is also recursive.

- $L1$ says n no. of a 's followed by n no. of b 's followed by n no. of c 's. $L2$ says m no. of d 's followed by m no. of e 's followed by m no. of f 's.
- Their concatenation first matches no. of a 's, b 's and c 's and then matches no. of d 's, e 's and f 's. So it can be decided by TM.

Properties of Recursive and RELs

Kleene Closure:

If L_1 is recursive, its kleene closure L_1^* will also be recursive.

For Example:

$$L_1 = \{a^n b^n c^n \mid n \geq 0\}$$

$$L_1^* = \{a^n b^n c^n \mid n \geq 0\}^* \text{ is also recursive.}$$

Properties of Recursive and RELs

Intersection and complement:

If L_1 and L_2 are two recursive languages, their intersection $L_1 \cap L_2$ will also be recursive.

For Example:

$$L_1 = \{a^n b^n c^n d^m \mid n \geq 0 \text{ and } m \geq 0\}$$

$$L_2 = \{a^n b^n c^n d^n \mid n \geq 0 \text{ and } m \geq 0\}$$

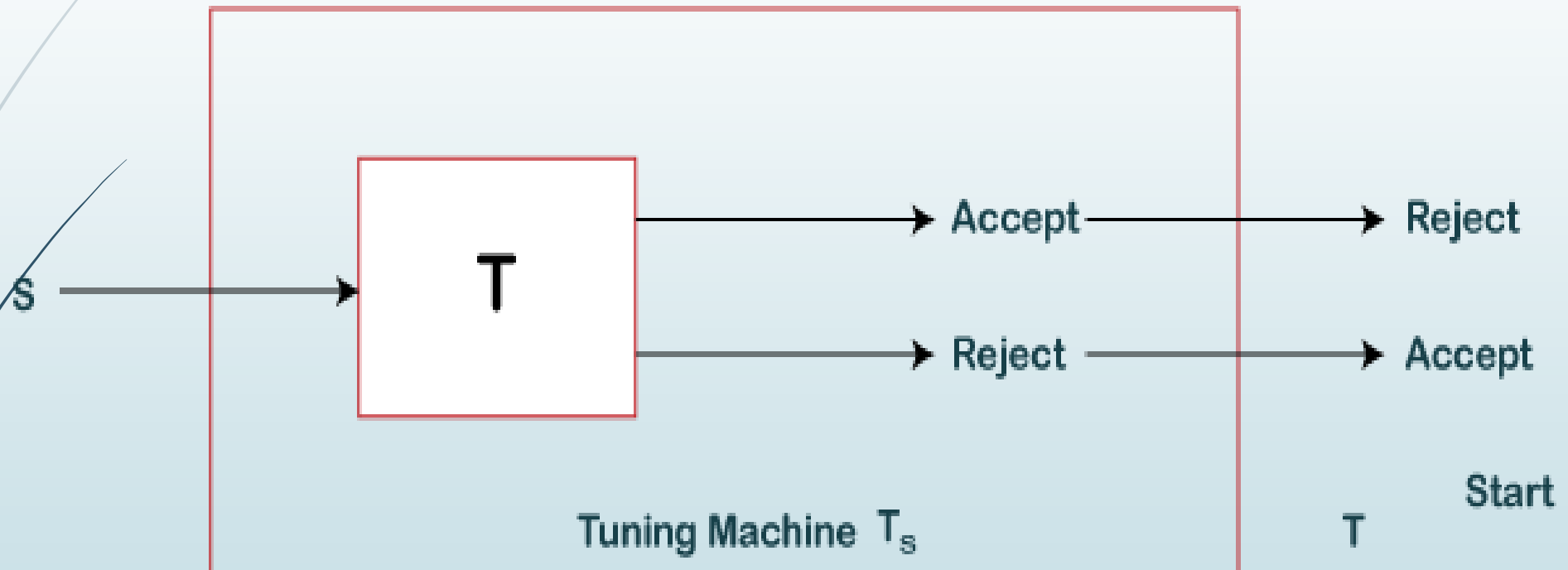
$$L_3 = L_1 \cap L_2$$

$$= \{a^n b^n c^n d^n \mid n \geq 0\} \text{ will be recursive.}$$

- L_1 says n no. of a 's followed by n no. of b 's followed by n no. of c 's and then any no. of d 's. L_2 says any no. of a 's followed by n no. of b 's followed by n no. of c 's followed by n no. of d 's.
- Their intersection says n no. of a 's followed by n no. of b 's followed by n no. of c 's followed by n no. of d 's. So it can be decided by Turing machine, hence recursive.
- Similarly, complement of recursive language L_1 which is $\Sigma^* - L_1$, will also be recursive.

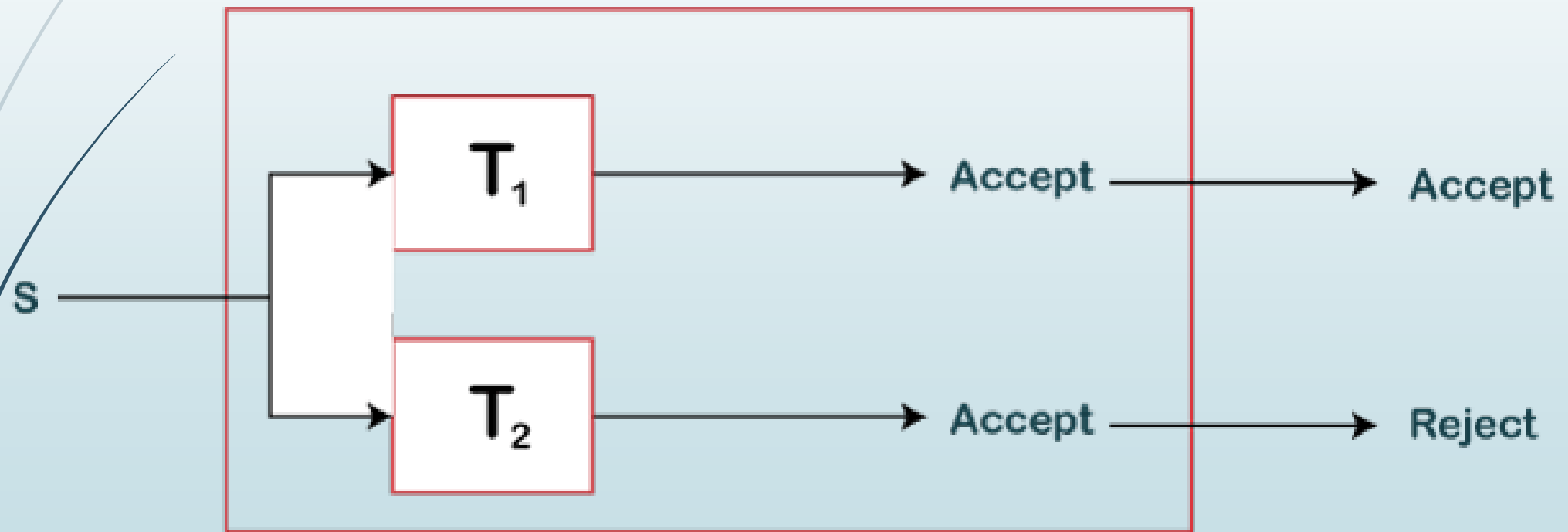
Properties of Recursive and RELs

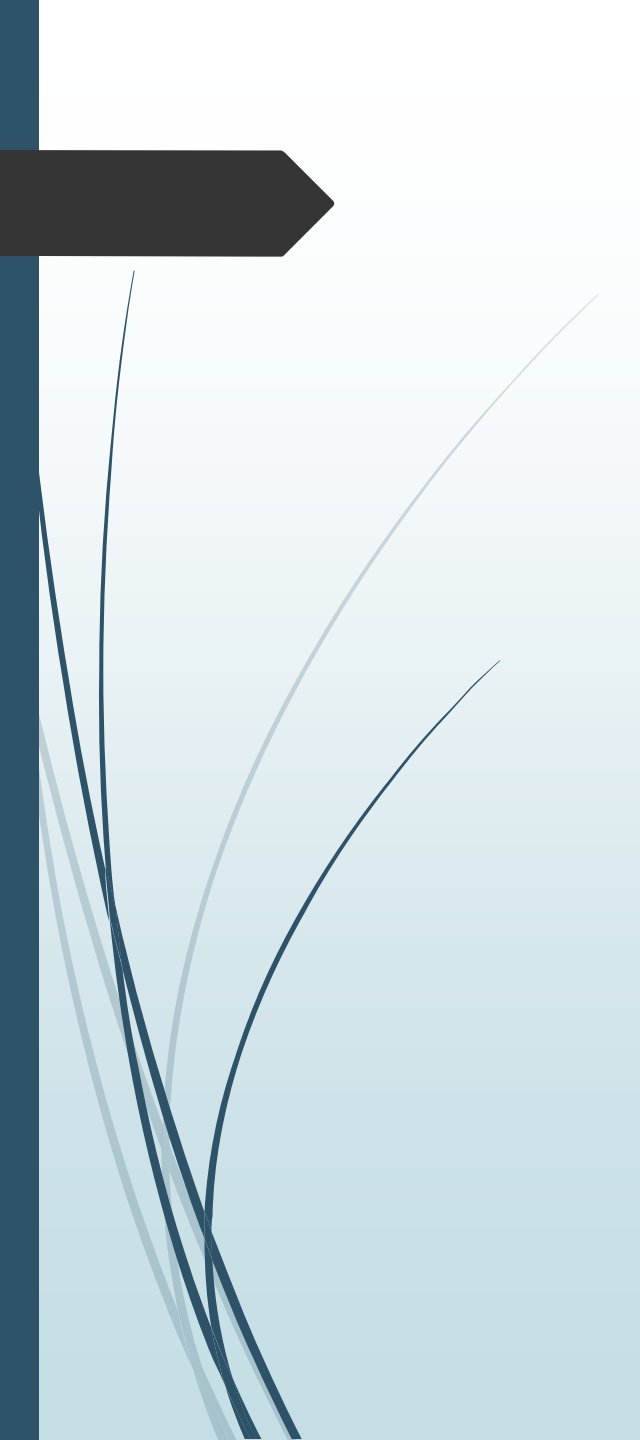
Complement : If L is recursive language then its complement L' is also recursive language



Properties of Recursive and RELs

Complement : L is a language and its complement L' is recursively enumerable language, then L is also a recursive language.





Questions ?